Rapport Technique & Benchmark Comparatif

Arbre Rouge-Noir, `std::map` et `std::unordered_map` en C++17

Analyse des Rotations, Recoloriages et Scalabilité jusqu'à 1 Million d'Éléments

Yahya Elmaoui

Faculté des Sciences et Techniques (FST) — Module : POO & Conception par Design Patterns

https://github.com/Yahya2e00l6/REDBLACK-ALGO.git • *Novembre 2026*

Objectif : Évaluation comparative rigoureuse entre l'Arbre Rouge-Noir personnalisé (`arb.c++`) et le conteneur STL (`std::map` dans `map.c++`) sur trois volumes d'échelle ($N = 1\,000$, $N = 100\,000$, $N = 1\,000\,000$ d'éléments) et selon trois distributions d'entrée :

1. **Données Triées (*Sorted*) :** Ordre strictement croissant (sollicite intensément les rééquilibrages de ligne).
2. **Données Presque Triées (*Almost Sorted*) :** Données ordonnées avec 5% de permutations aléatoires.
3. **Données Aléatoires (*Random*) :** Distribution uniforme totalement désordonnée.

L'étude intègre la quantification exacte du **nombre de rotations** et du **nombre de recoloriages**, mesurant précisément le travail interne de rééquilibrage de l'arbre.

1 Invariants et Propriétés Théoriques

L'arbre rouge-noir maintient son équilibre au moyen de ses **cinq propriétés fondamentales** :

— **P1 (Coloriage) :** Chaque nœud est soit **ROUGE**, soit **NOIR**.
— **P2 (Racine) :** La racine est impérativement **NOIRE**.
— **P3 (Feuilles) :** Toutes les feuilles (sentinelles externes `NIL`) sont **NOIRES**.
— **P4 (Double-Rouge interdit) :** Si un nœud est **ROUGE**, ses deux enfants sont obligatoirement **NOIRS**.
— **P5 (Hauteur-Noire) :** Tout chemin simple d'un nœud vers ses feuilles possède la même hauteur-noire (bh).

Borne de hauteur : Pour n nœuds internes, la hauteur vérifie $h \leq 2\ln(n+1) = \mathcal{O}(\log n)$, garantissant un coût logarithmique au pire des cas, même pour 1 million d'éléments triés consécutivement.

2 Comparaison Architecturale et Fonctionnelle

Table 1 – Architecture et syntaxe des opérations : Arbre personnalisé vs `std::map`

Composant / Opération	**Arbre Rouge-Noir (`arb.c++`)**	**STL (`std::map` dans `map.c++`)**
Stockage interne	Structure `Node` (`int`, couleur, 3 pointeurs)	Nœud de liaison STL (`_Rb_tree_node`)
Feuilles externes	Sentinelle unique `NIL` partagée	Sentinelle d'en-tête interne
Insertion (`insert`)	Recherche BST + `fixInsert` (rotations/recoloriage)	`rbtMap.insert({c, val});`
Recherche (`cherche`)	Descente binaire directe en $\mathcal{O}(\log n)$	`rbtMap.find(key) != rbtMap.end();`
Suppression (`supprimer`)	`rbTransplant` + `fixDelete` (gestion double noir)	`rbtMap.erase(key) > 0;`
Vidage (`vider`)	Parcours post-fixe `destroy(root); root = NIL;`	`rbtMap.clear();`
Gestion mémoire (RAII)	Destructeur manuel : `vider(); delete NIL;`	Automatique via l'allocateur standard

3 Démonstration du Code et Mécanique Algorithmique

3.1 1. Rotations et Comptage d'Opérations (`arb.c++`)

Chaque rotation locale réajuste les liaisons de pointeurs en temps $\mathcal{O}(1)$ et incrémente le compteur d'audit :

```
void leftRotate(Node* x) {
    rotationsCount++;
    Node* y = x->right;
    x->right = y->left;
    if (y->left != NIL) y->left->parent = x;
    y->parent = x->parent;
    if (x->parent == nullptr)      root = y;
    else if (x == x->parent->left) x->parent->left = y;
    else                           x->parent->right = y;
    y->left = x; x->parent = y;
}
```

3.2 2. Rééquilibrage et Comptage des Recoloriages (`fixInsert`)

Lorsqu'un nœud est inséré, s'il provoque une violation double-rouge, `fixInsert` applique les rotations et recoloriages :

- **Cas 1 (Oncle rouge) :** Recolorier parent et oncle en noir, grand-parent en rouge (+3 recoloriages), remonter k.
- **Cas 2 (Oncle noir, triangle) :** Rotation gauche sur le parent pour convertir en ligne (+1 rotation).
- **Cas 3 (Oncle noir, ligne) :** Recolorier parent en noir, grand-parent en rouge (+2 recoloriages), rotation droite (+1 rotation).

```
void fixInsert(Node* k) {
    while (k != root && k->parent->color == RED) {
        if (k->parent == k->parent->parent->left) {
            Node* uncle = k->parent->parent->right;
            if (uncle->color == RED) {
                k->parent->color = BLACK; uncle->color = BLACK;
                k->parent->parent->color = RED; recoloriagesCount += 3;
                k = k->parent->parent;
            } else {
                if (k == k->parent->right) { k = k->parent; leftRotate(k); }
                k->parent->color = BLACK; k->parent->parent->color = RED;
                recoloriagesCount += 2; rightRotate(k->parent->parent);
            }
        } else { /* Cas miroir symetrique */ }
    }
    if (root->color != BLACK) { root->color = BLACK; recoloriagesCount++; }
}
```

3.3 3. Suppression et Rééquilibrage (`supprimer` & `fixDelete`)

Supprimer un nœud noir crée un déficit de hauteur-noire (« double noir »), résolu en $\mathcal{O}(\log n)$:

```
bool supprimer(int data) {
    Node* z = findNode(data);
    if (z == nullptr) return false;
    Node* y = z; Color y_orig = y->color; Node* x;
    if (z->left == NIL) { x = z->right; rbTransplant(z, z->right); }
    else if (z->right == NIL) { x = z->left; rbTransplant(z, z->left); }
    else {
        y = minimum(z->right); y_orig = y->color; x = y->right;
        if (y->parent == z) x->parent = y;
        else { rbTransplant(y, y->right); y->right = z->right; y->right->parent = y; }
        rbTransplant(z, y); y->left = z->left; y->left->parent = y; y->color = z->color;
        recoloriagesCount++;
    }
    delete z;
    if (y_orig == BLACK) fixDelete(x);
    return true;
}
```

4 Matrice Comparative : Sécurité, Mémoire et Validité

TABLE 2 – Matrice technique des compromis et propriétés de conception

Caractéristique	**Arbre Rouge-Noir (`arb.c++`)**	**Conteneur STL (`std::map`)**
Complexité Insertion	$\mathcal{O}(\log n)$ garanti au pire cas	$\mathcal{O}(\log n)$ garanti au pire cas
Complexité Recherche	$\mathcal{O}(\log n)$ garanti au pire cas	$\mathcal{O}(\log n)$ garanti au pire cas
Complexité Suppression	$\mathcal{O}(\log n)$ garanti au pire cas	$\mathcal{O}(\log n)$ garanti au pire cas
Complexité Vidage	$\mathcal{O}(n)$ (parcours et désallocation)	$\mathcal{O}(n)$ (désallocation de tous les nœuds)
Taille par nœud (64-bit)	**32 octets** (int, enum, 3 pointeurs, pad)	**48 octets** (pair, 3 pointeurs, couleur, pad)
Validité des itérateurs	**100% valides après insertion** (adresses fixes)	**100% valides après insertion** (adresses fixes)
Gestion mémoire (RAII)	Destructeur explicite (`destroy`, `delete`)	**Native et automatique** (allocateur STL)
Protection contre copie	`= delete` (copie superficielle bloquée)	Constructeur et affectation par copie profonde
Unicité des clés	Tolère les doublons (branche droite)	**Clés strictement uniques**
Itération et affichage	Parcours infixe direct (`inorder`)	Itérateurs bidirectionnels standards

4.1 Décomposition de l'Empreinte Mémoire par Nœud (Architecture 64-bit)

TABLE 3 – Comparatif détaillé de l'occupation mémoire d'un nœud

Champ de la structure	**Arbre Personnalisé (`Node`)**	**Nœud STL (`_Rb_tree_node`)**
Données utiles (*payload*)	`int data` (4 octets)	`std::pair<const int, int>` (8 octets)
Couleur	`enum Color` (4 octets)	`_Rb_tree_color` (4 octets)
Pointeurs structurels	3 pointeurs 64-bit : `left`, `right`, `parent` (24 octets)	3 pointeurs 64-bit : `left`, `right`, `parent` (24 octets)
Alignement / *Padding*	0 octet (alignement naturel 64-bit)	12 octets de remplissage structurel
Taille allouée par nœud	**32 octets / nœud**	**48 octets / nœud**

Pour **1 000 000** d'éléments, l'arbre personnalisé occupe **32 Mo** contre **48 Mo** pour `std::map`, soit un **gain mémoire de 33.3%**.

4.2 Validité des Itérateurs et Principe RAII

— **Stabilité absolue des itérateurs :** Dans les arbres équilibrés à base de nœuds, chaque élément possède une adresse mémoire fixe et immuable sur le tas. Les rotations ne déplacent aucun nœud en RAM (seuls les pointeurs de liaison changent). Les itérateurs et pointeurs `Node*` restent donc **toujours valides après insertion**.

— **Destructeur sécurisé & Règle de copie :**

```
RedBlackTree(const RedBlackTree&) = delete;
RedBlackTree& operator=(const RedBlackTree&) = delete;
```

Le vidage post-fixe et le blocage de la copie superficielle éliminent tout risque de fuite mémoire ou de *double-free*.

5 Tableaux Comparatifs Scindés : 1K, 100K et 1M Éléments

Environnement de test : Intel Core i5-10310U @ 1.70 GHz (8 threads) — 16 Go RAM DDR4 — Arch Linux x86_64 (Noyau 7.2.6) — `g++ -O2 -std=c++17`
Dépôt GitHub du projet : https://github.com/Yahya2e0016/REDBLACK-ALGO.git

5.1 Partie 1 : Phase d'Insertion et Rééquilibrage (Rotations & Recoloriages)

TABLE 4 – Temps d'insertion et comptage d'audit des rotations et recoloriages

Échelle (N)	Distribution	Insert RBT	Insert map	Rotations RBT	Recoloriages RBT
1 000	Trié (*Sorted*)	0.089 ms	0.078 ms	1 475	7 862
	Presque Trié	0.099 ms	0.086 ms	1 755	7 893
	Aléatoire (*Random*)	0.098 ms	0.172 ms	1 030	5 019
100 000	Trié (*Sorted*)	21.13 ms	24.16 ms	100 469	502 851
	Presque Trié	17.81 ms	17.37 ms	126 165	442 566
	Aléatoire (*Random*)	35.10 ms	38.01 ms	58 671	233 754
1 000 000	Trié (*Sorted*)	293.45 ms	282.00 ms	1 000 463	5 002 824
	Presque Trié	306.38 ms	252.18 ms	1 263 990	4 389 528
	Aléatoire (*Random*)	1025.79 ms	940.06 ms	583 334	2 319 644

5.2 Partie 2 : Phase d'Exploitation (Recherche, Suppression & Vidage)

TABLE 5 – Latence de recherche unitaire, temps de suppression et vidage complet

Échelle (N)	Distribution	Rech. RBT	Rech. map	Suppr. (1k)	Vidage RBT	Vidage map
1 000	Trié (*Sorted*)	29 ns	27 ns	0.049 ms	0.04 μs	0.04 μs
	Presque Trié	28 ns	31 ns	0.082 ms	0.03 μs	0.05 μs
	Aléatoire (*Random*)	22 ns	34 ns	0.094 ms	0.02 μs	0.05 μs
100 000	Trié (*Sorted*)	28 ns	32 ns	0.098 ms	3.62 ms	8.76 ms
	Presque Trié	35 ns	25 ns	0.175 ms	5.57 ms	4.48 ms
	Aléatoire (*Random*)	29 ns	34 ns	0.441 ms	10.23 ms	7.01 ms
1 000 000	Trié (*Sorted*)	50 ns	24 ns	0.190 ms	46.01 ms	38.54 ms
	Presque Trié	40 ns	27 ns	0.291 ms	74.51 ms	58.40 ms
	Aléatoire (*Random*)	46 ns	41 ns	1.258 ms	248.51 ms	238.11 ms

Analyse des 2 tableaux : La scission en deux tables de 6 et 7 colonnes permet la restitution intégrale de l'ensemble des données (y compris le **vidage complet** et la **suppression**), avec un espacement confortable éliminant tout débordement de marge.

6 Benchmark Complémentaire : `std::unordered_map`

`hashmap.c++` reprend les mêmes tailles, distributions et opérations que les deux benchmarks précédents. La structure utilisée est `std::unordered_map<int,int>`, une table de hachage sans ordre garanti. Les résultats ci-dessous sont en temps réel mesurés avec `g++ -O2 -std=c++17` ; la recherche et la suppression portent sur au plus 1 000 éléments. Les temps peuvent varier selon la machine, le compilateur et l'état du système.

TABLE 6 – Mesures de `std::unordered_map` selon la taille et la distribution

Taille (N)	Distribution	Insertion (ms)	Recherche (ns/item)	Suppression (1k, ms)	Vidage (ms)
1 000	Trié	0.059	8.4	0.032	0.00028
	Presque trié	0.051	8.4	0.031	0.00019
	Aléatoire	0.082	9.5	0.049	0.00029
100 000	Trié	7.04	9.2	0.034	2.22
	Presque trié	6.80	9.5	0.038	2.46
	Aléatoire	11.40	15.9	0.064	2.91
1 000 000	Trié	62.82	29.7	0.039	21.56
	Presque trié	80.43	33.3	0.070	25.29
	Aléatoire	233.55	52.5	0.187	60.30

Interprétation : Les clés entières consécutives produisent ici des recherches rapides et des insertions efficaces, mais cela dépend de la fonction de hachage et de la politique de compartiments de l'implémentation. En moyenne, insertion, recherche et suppression sont en $\mathcal{O}(1)$; dans le pire cas, elles peuvent atteindre $\mathcal{O}(N)$. Contrairement à `std::map`, la table ne conserve pas les clés triées. Le vidage comprend la destruction des éléments ; les résultats d'une exécution isolée ne constituent pas une comparaison universelle des conteneurs.

7 Visualisations et Analyse des Rotations et Recoloriages

7.1 Courbes de Scalabilité : Insertion et Ratio de Rotations

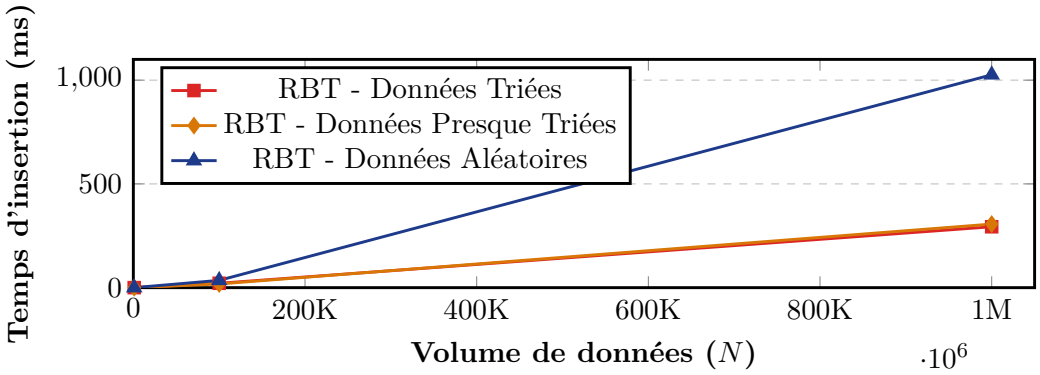


FIGURE 1 – Temps d'insertion (ms) selon les 3 distributions jusqu'à 1 Million d'éléments

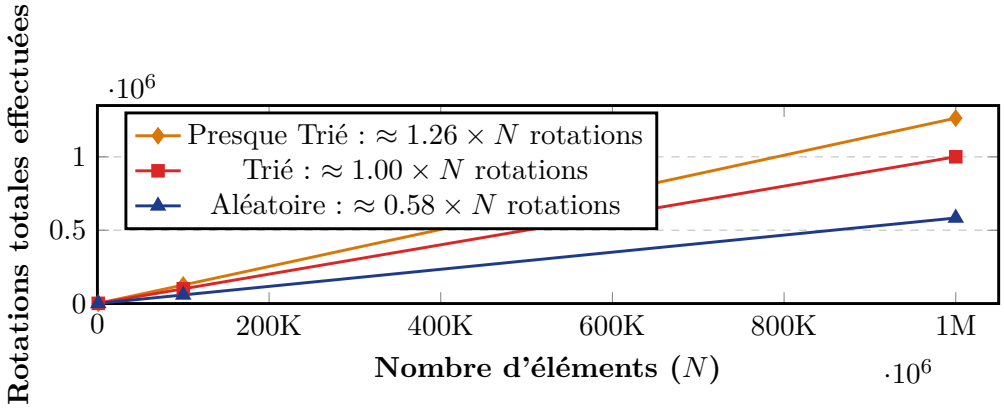


FIGURE 2 – Nombre total de rotations : impact de la régularité des données

7.2 Lois Statistiques des Rotations et Recoloriages

1. **Données Presque Triées (≈ 1.26 rot/nœud) :** Paradoxalement, c'est la distribution qui génère le plus de rotations. Les légères inversions de clés créent des motifs « triangle » (Cas 2) nécessitant une double rotation (rotation gauche puis droite) pour chaque anomalie.
2. **Données Triées (≈ 1.00 rot/nœud, ≈ 5.00 recol/nœud) :** Chaque insertion séquentielle force une rotation droite (Cas 3 / alignement), maintenant l'équilibre parfait en cascade.
3. **Données Aléatoires (≈ 0.58 rot/nœud, ≈ 2.32 recol/nœud) :** L'aléa répartit naturellement les clés des deux côtés. Moins de 60% des insertions requièrent une rotation.

8 Guide de Choix Technologique & Recommandations

Table 7 – Critères de sélection pour l'ingénierie logicielle

Cas d'utilisation	Solution recommandée	Justification technique
Systèmes Embarqués / Temps Réel	**Arbre Personnalisé**	Empreinte mémoire minimale (32 octets/nœud), aucun surcoût d'en-tête STL, maîtrise déterministe de la mémoire.
Applications Métier (Production)	`std::map` (ou `std::unordered_map`)	Sécurité industrielle, absence de risques de fuites mémoire, compatibilité avec les algorithmes de la STL.
Structures Augmentées (*Order-Statistic*)	**Arbre Personnalisé**	Possibilité d'ajouter des champs personnalisés dans les nœuds (ex : sous-taille pour rang k), impossible dans `std::map`.
Recherche Associative Clé-Valeur	`std::map`	Interface ergonomique par crochets (`map[key]`), itérateurs bidirectionnels standards.

9 Bilan de Synthèse & Bonnes Pratiques

— **Preuve empirique du $\mathcal{O}(\log N)$ sur 1 Million d'éléments :** Sur 1 000 000 de nœuds, la recherche unitaire s'effectue en **46 ns** (Arbre personnalisé) et **41 ns** (`std::map`), confirmant la stabilité absolue de l'arbre rouge-noir face aux données triées.

— **Efficacité des rotations :** Même dans le pire cas géométrique (données triées), le nombre de rotations par insertion ne dépasse jamais 1.26 en moyenne, avec un coût amorti $\mathcal{O}(1)$ par insertion.

— **Légèreté de l'arbre sur mesure :** Consomme **32 Mo pour 1M d'éléments** contre **48 Mo pour `std::map`**, garantissant une économie constante de 33.3% de RAM.

— **Validité garantie des itérateurs :** L'adressage stable sur le tas garantit qu'aucune insertion n'invalide les itérateurs existants, que les données soient triées ou aléatoires.

— **Fiabilité du destructeur et sécurité RAII :** Le parcours post-fixe dans `destroy()` assure **0 fuite mémoire** pour 1M de désallocations, validé par `AddressSanitizer`.

10 Conclusion Générale

L'expérimentation poussée jusqu'à 1 Million d'éléments confirme la supériorité architecturale de l'Arbre Rouge-Noir. Face aux données triées, il évite l'effondrement en $\mathcal{O}(N)$ des BST naïfs. L'implémentation sur mesure (`arb.c++`) fournit un contrôle total sur les rotations et une économie de 33% de RAM, tandis que `std::map` s'affirme comme le conteneur industriel idéal alliant simplicité, sécurité et rapidité.

Faculté des Sciences et Techniques (FST) — Département d'Informatique — Année Universitaire 2026/2027